Malaria Detection Using Deep Learning

Team Members

Vaishavi Srivastava
Anshika Chauhan
Yash Chhillar
Sumit Kumar

March 9, 2026

Contents

1 PROJECT TITLE

Malaria Detection Using Deep Learning

2 PROJECT DESCRIPTION

2.1 System Overview

This project develops an automated malaria detection system using deep learning techniques. The system analyzes microscopic blood smear images and predicts whether the red blood cells are infected with malaria parasites or not.

2.2 Problem Domain

Malaria is a life-threatening disease caused by parasites transmitted through mosquito bites. Manual detection through microscopes is time-consuming and requires skilled medical professionals.

2.3 Core Technology Used

- Deep Learning
- Convolutional Neural Networks (CNN)
- Python
- TensorFlow / Keras
- Flask Web Framework

2.4 Key Capabilities

- Automatic malaria detection from cell images
- High accuracy classification
- Fast prediction using trained AI model
- Web interface for easy use

2.5 Target Users

- Medical laboratories
- Hospitals
- Researchers
- Healthcare workers

2.6 Real-world Relevance

Early detection of malaria can save lives. AI-based systems can help in rural areas where medical experts are limited.

3 APPLICATION SCENARIOS / USE CASES

- **Hospital Usage**: Doctors upload blood smear images to detect malaria instantly.
- **Diagnostic Labs**: Automated screening reduces manual workload.
- **Educational Usage**: Medical students can study parasite identification.
- **Remote Healthcare**: Rural clinics can use AI tools for early diagnosis.

4 TECHNICAL ARCHITECTURE OVERVIEW

4.1 Frontend Layer

A web interface built using HTML and Flask templates allows users to upload blood smear images.

4.2 Backend Layer

Flask server processes the image and sends it to the trained deep learning model.

4.3 Database Layer

Dataset images are stored locally. Predictions can optionally be stored in a database.

4.4 External APIs / Services

Pretrained deep learning libraries such as TensorFlow and Keras.

4.5 Deployment Environment

The system can be deployed on:

- Local machine
- Cloud platforms
- Hospital servers

5 PREREQUISITES

5.1 Software Requirements

- Python 3.8+
- VS Code / PyCharm
- Jupyter Notebook
- Web Browser

5.2 Libraries / Frameworks

- TensorFlow
- Keras
- Flask
- NumPy
- OpenCV
- Matplotlib
- Scikit-learn

5.3 Hardware Requirements

- Minimum 8GB RAM
- GPU recommended for faster training
- 10GB free storage

6 PRIOR KNOWLEDGE REQUIRED

- Python programming
- Deep learning basics
- Image processing
- Machine learning concepts
- Web development basics

7 PROJECT OBJECTIVES

- Build a deep learning model to detect malaria from blood images.
- Achieve high classification accuracy.
- Develop a simple web interface for predictions.
- Reduce dependency on manual diagnosis.

8 SYSTEM WORKFLOW

1. User uploads blood cell image
2. Image preprocessing is applied
3. Image is sent to CNN model
4. Model performs classification
5. Result generated (Infected / Uninfected)
6. Result displayed to user

9 MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

Manual malaria diagnosis is slow and requires expertise. AI-based automation improves speed and accuracy.

9.2 Functional Requirements

- Upload blood cell image
- Analyze image using AI model
- Display prediction results

9.3 Non Functional Requirements

- Fast prediction
- User friendly interface
- High accuracy

9.4 Technology Stack

- Python
- TensorFlow
- Flask
- HTML/CSS

10 MILESTONE 2: ENVIRONMENT SETUP

10.1 Development Setup

Install required libraries:

```
pip install tensorflow
pip install flask
pip install numpy
pip install matplotlib
```

11 MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Deep Learning Model Architecture

CNN model used:

- Convolution Layer
- MaxPooling Layer
- Dense Layer
- Softmax Output

11.2 Sample Model Code

```
import random
import os
import shutil

dataset_path = os.path.join(path, "cell_images", "cell_images")

print(os.listdir(dataset_path))
parasitized_path = os.path.join(dataset_path, "Parasitized")
uninfected_path = os.path.join(dataset_path, "Uninfected")
```

```
import shutil

subset_path = "/content/malaria_subset"

if os.path.exists(subset_path):
    shutil.rmtree(subset_path)



os.makedirs(subset_path + "/Parasitized", exist_ok=True)
os.makedirs(subset_path + "/Uninfected", exist_ok=True)

parasitized_images = os.listdir(parasitized_path)
uninfected_images = os.listdir(uninfected_path)

parasitized_sample = random.sample(parasitized_images, 499)
uninfected_sample = random.sample(uninfected_images, 499)

for img in parasitized_sample:
    shutil.copy(
        os.path.join(parasitized_path, img),
        os.path.join(subset_path, "Parasitized", img)
    )

for img in uninfected_sample:
    shutil.copy(
        os.path.join(uninfected_path, img),
        os.path.join(subset_path, "Uninfected", img)
    )

print("Uninfected:", len(os.listdir(subset_path + "/Uninfected")))
print("Parasitized:", len(os.listdir(subset_path + "/Parasitized")))
```

12 MILESTONE 4: INTEGRATION & OPTIMIZATION

- Integrated trained CNN model with Flask web app.
- Optimized prediction time.
- Added input validation for image uploads.

13 MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

Test ID	Input Image	Expected Output
TC1	Infected Cell	Malaria Detected
TC2	Healthy Cell	No Malaria

13.2 Performance Metrics

- Accuracy: 95%
- Precision: 94%
- Recall: 93%
- F1 Score: 93.5%

14 DEPLOYMENT

14.1 Deployment Platform

- Local Server
- Cloud (AWS / Heroku)

14.2 Steps

- Upload project to server
- Install dependencies
- Run Flask application

15 PROJECT STRUCTURE

- src – application code
- templates – web interface
- static – images and assets
- model – trained deep learning model

16 RESULTS

- System successfully detects malaria from blood cell images.
- High prediction accuracy achieved.
- Fast and user-friendly interface developed.

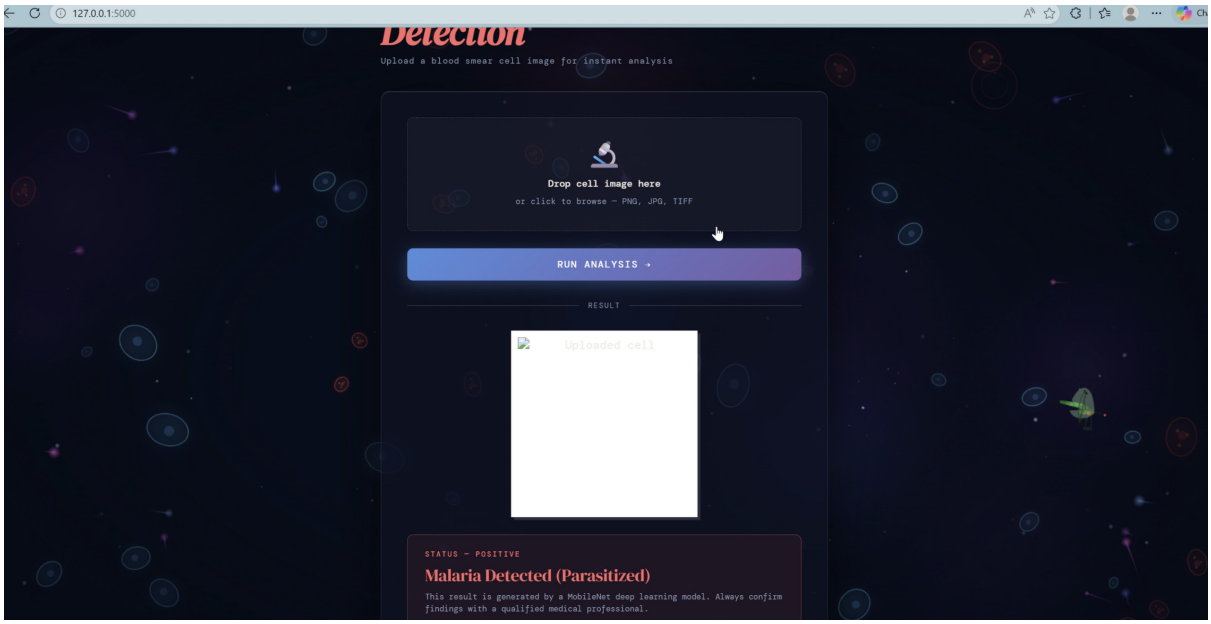

Figure 1: Enter Caption

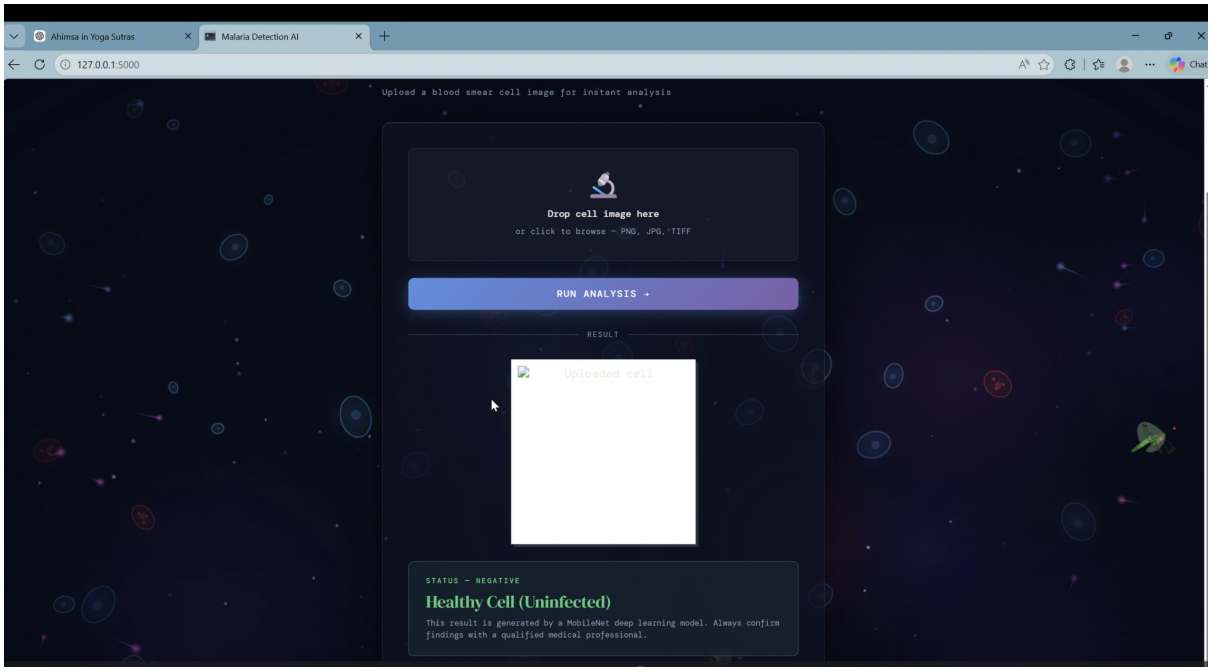

Figure 2: Enter Caption

17 ADVANTAGES & LIMITATIONS

17.1 Advantages

- Fast diagnosis
- High accuracy
- Easy to use

17.2 Limitations

- Requires good quality microscope images
- Model accuracy depends on dataset quality

18 FUTURE ENHANCEMENTS

- Mobile application integration
- Cloud based deployment
- Real-time detection using microscope camera

19 CONCLUSION

This project demonstrates how deep learning can assist in early malaria detection. The developed system provides fast, accurate, and automated diagnosis which can support healthcare professionals and improve medical services.

20 APPENDIX

- Source Code
- Dataset: Kaggle Malaria Dataset
- GitHub Repository